Checkpoint 2 Operating Script

Sierra-Lima slice: `restaurant-service` + `menu-service`
Group 7, Sten-Qy-Li

2026-05-12, 14:15–14:30 EET (15 minutes)

1 Pre-flight checklist

Open these BEFORE the instructor sits down. Each item: open the app, verify it is ready as described.

- **Docker Desktop** – ready when the whale icon in the system tray stops animating and the Dashboard reads *Engine running*.
- **IntelliJ IDEA** – open the project at `C:\MSc-Computer-Science\Semester-2\esi\esi`; ready when the Project tool window shows the seven service folders and no progress bar runs at the bottom.
- **Windows Terminal / PowerShell** – one window, working directory set to the project root: `cd C:\MSc-Computer-Science\Semester-2\esi\esi`.
- **A browser** – one fresh window with no tabs (open new tabs during scenes).

Pre-warm the stack 10 minutes before the meeting starts so the first-time Maven + npm build does not eat your demo time:

DO

In the PowerShell window opened above:

```
git checkout checkpoint-2
docker compose up -d --build
```

SEE

First-time build takes 3–5 minutes. Run `docker compose ps` repeatedly until all six services show `(healthy)`. Subsequent runs after that come up in under a minute.

Already done by tagging this release

`checkpoint-2` tag exists on `origin/sten` pointing at commit `ee51ceb`.

All Sierra-Lima changes committed and pushed to `origin/sten`; working tree is clean.

Group 7 demo slot for 14:15–14:30 on 2026-05-12 confirmed in the Moodle poll.

2 Setup

(In the PowerShell terminal opened in pre-flight, project root.)

DO

```
git checkout checkpoint-2
```

SEE

```
HEAD is now at ee51ceb Add Checkpoint 2 runtime stack to sten
```

DO

```
docker compose ps
```

SEE

All six containers show `Up X (healthy)`: `restaurant-db`, `menu-db`, `restaurant-service`, `menu-service`, `gateway`, `frontend`.

screenshots/setup-1.png

Figure 1: `docker compose ps` output: all six services healthy.

3 Demonstration scenes

3.1 Scene 1 – Item A: second service runs (4 points)

SAY

"Both my services start from one compose file with their own Postgres databases."

DO

In the browser, open these three tabs (one at a time, one per Ctrl+T):

```
http://localhost:8081/swagger-ui.html
http://localhost:8082/swagger-ui.html
```

Then back in PowerShell, run:

```
curl.exe -s "http://localhost:8080/restaurants?size=2"
```

And:

```
curl.exe -s "http://localhost:8080/restaurants/d0000001-0000-0000-0000-000000000001/menu-items"
```

SEE

Tab 1 (`:8081`): the restaurant-service Swagger UI lists six endpoints under `restaurant-controller` (POST/GET list/GET id/PUT/PATCH status/GET availability), covering R19 register/manage and R20 open-closed status.
Tab 2 (`:8082`): the menu-service Swagger UI lists six endpoints under `menu-controller` (POST/GET list-by-restaurant/GET id/PUT/DELETE/POST validate), covering R21 owner-gated CRUD and R22 browse menu.
The first `curl.exe` prints a paged JSON with `"totalElements":6` – the six restaurants seeded by Flyway migration `V2__seed_demo_data.sql` into `restaurant-db`. The second `curl.exe` prints a JSON array with four menu items for Pizza Antonio (`d0000001`), seeded into `menu-db`.

screenshots/scene1-step1.png

Figure 2: restaurant-service Swagger UI on `:8081` listing the six endpoints.

screenshots/scene1-step2.png

Figure 3: menu-service Swagger UI on `:8082` listing the six endpoints.

screenshots/scene1-step3.png

Figure 4: PowerShell `curl.exe` output: paged JSON of six restaurants from `restaurant-db`, followed by the JSON array of four menu items from `menu-db`.

PROVES

[BASE] "Second Responsibility: Second service OR integration component runs." *(4 points; Project2026.pdf p. 4.)* The same scene also satisfies the three Checkpoint-1 sub-criteria the brief explicitly inherits for the second service via the IMPORTANT note ("just implement the points A, B, and D applied to service implementation in Checkpoint 1"):

- "A. Running Service: Service starts and endpoints are accessible." (`docker compose ps` shows both services healthy; their endpoints answer.)
- "B. API Implementation: All endpoints implemented according to Assignment 3." (Six endpoints per service in Swagger, matching the R19–R22 surface from Assignment 3.)
- "D. Persistence: Database connected, and some data stored and retrieved." (Six restaurants and four menu items returned from real Postgres via Flyway migrations.)

3.2 Scene 2 – Item B: live cross-service hop, real call not mocked (2 points)

SAY

"Now I'll show menu-service calling restaurant-service over real HTTP for an ownership check."

DO

In PowerShell, run the positive case (a JWT for the owner of restaurant `d0000001`, pre-minted with the dev secret from `application.properties`):

```
curl.exe -i -X POST ‘
  "http://localhost:8080/restaurants/d0000001-0000-0000-0000-000000000001/menu-items" ‘
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
    ↪ eyJpc3MiOiJxdWlja2JpdGUtdXNlci1zZXJ2aWNlIiwic3ViIjoiMDAwMDAwMDAtMDAwMC0wMDAwLTAwMDAtMDAwMDAwMDAwMDAxIiwid
    ↪ .iQUGu1X-tIAFVEtAUM4BMINT5RJ0zPVvY-s8pHElu48" ‘
  -H "Content-Type: application/json" ‘
  --data-raw ’{"name":"Demo Hop","priceAmount":3.33,"priceCurrency":"EUR","category":"Main"}’
```

SEE

First line: `HTTP/1.1 201 Created`. Body is the new menu item with a fresh `menuItemId` and `restaurantId":"d0000001-0000-0000-0000-000000000001"`.

screenshots/scene2-step1.png

Figure 5: Positive case: HTTP 201 with the created menu-item JSON.

SAY

"And here's why the lookup must be real – a different owner's token gets rejected."

DO

In PowerShell, run the negative case (a JWT for a DIFFERENT user who owns other restaurants but not `d0000001`):

```
curl.exe -i -X POST ‘
  "http://localhost:8080/restaurants/d0000001-0000-0000-0000-000000000001/menu-items" ‘
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
    ↪ eyJpc3MiOiJxdWlja2JpdGUtdXNlci1zZXJ2aWNlIiwic3ViIjoiMDAwMDAwMDAtMDAwMC0wMDAwLTAwMDAtMDAwMDAwMDAwMDAyIiwid
    ↪ .KxoEi_WDtGFWzGky1q8nbw41ghr_zx0oZM16qNP02A8" ‘
  -H "Content-Type: application/json" ‘
  --data-raw ’{"name":"Should be rejected","priceAmount":1.00,"priceCurrency":"EUR","category":"
    ↪ Main"}’
```

SEE

First line: `HTTP/1.1 403 Forbidden`. Body contains `"User 00000000-0000-0000-0000-000000000002 does not own restaurant d0000001-0000-0000-0000-000000000001"`.
The `ownerId` that menu-service used to reject the request is not stored anywhere in `menu-db` – menu-service only keeps `restaurantId` as a UUID reference (no foreign key, no copy of ownership). The only way menu-service learned that ownerId is via the live HTTPS call to `restaurant-service:8081/restaurants/d0000001-....`

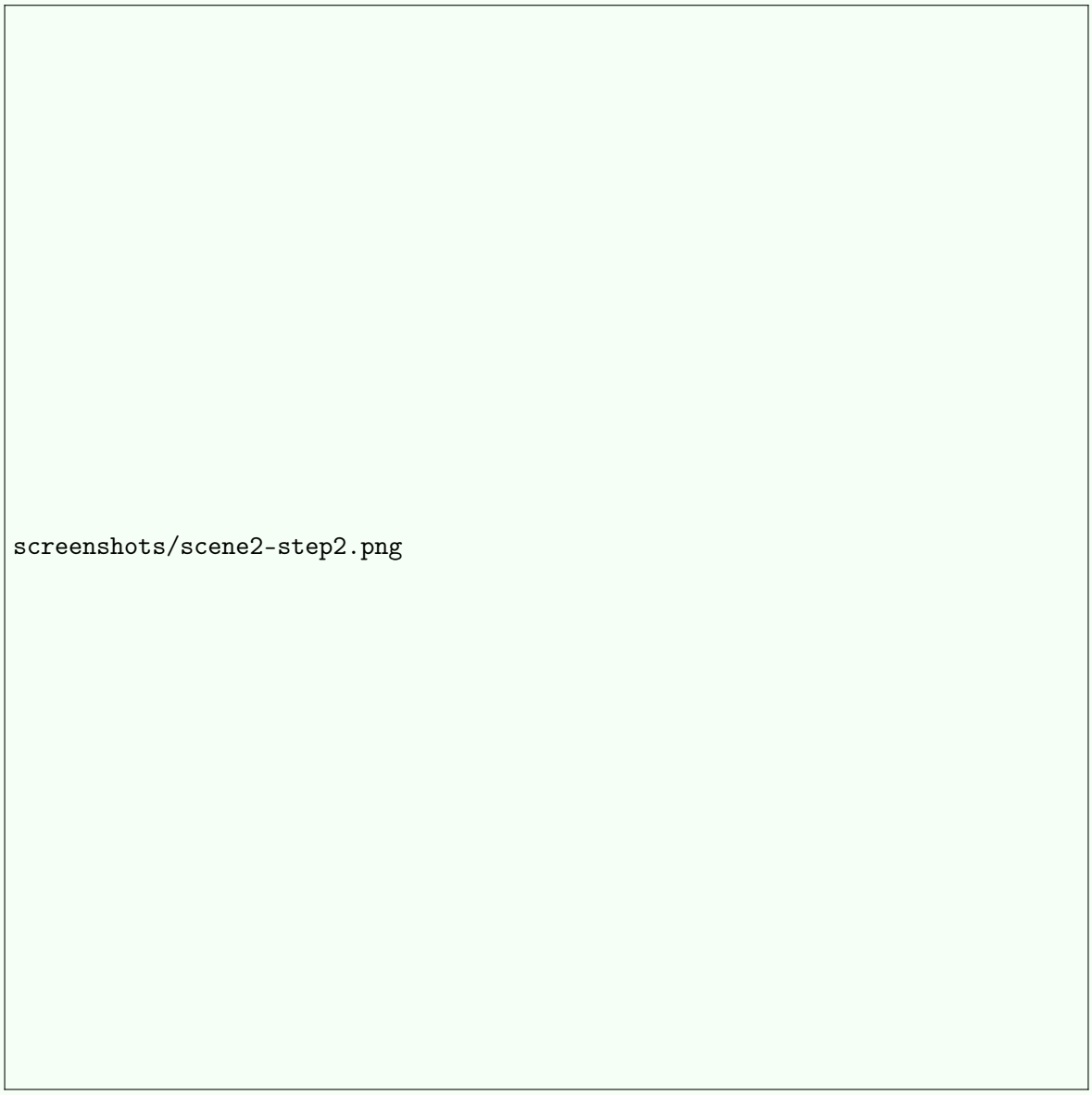

Figure 6: Negative case: HTTP 403 with the ownership-denial message containing the `ownerId` learned via the cross-service lookup.

PROVES

[BASE] "Basic Integration: At least one working interaction between two implemented services must be demonstrated (real call, not mocked)." *(2 points; Project2026.pdf p. 4.)* The 403 response contains an `ownerId` that menu-service does not persist locally; it could only have been learned through the live HTTP call to `restaurant-service`. The implementation is in `menu-service/src/main/java/ee/ut/esi/quickbite/menu/security/RestaurantOwnershipClient.java`.

3.3 Scene 3 – Item C: initial frontend (2 points)

SAY

"And here's the frontend, hitting both my services through the gateway."

DO

In the browser, open a new tab to `http://localhost:8090`. Open Chrome DevTools (`F12`) and switch to the Network tab. Then click **Restaurants** in the top nav.

SEE

The page renders six restaurant cards: Pizza Antonio (Tartu), Sushi Lumi (Tartu), Cafe Nero (Tartu), Burger Bros (Tallinn), Vegan Vibes (Tallinn), Pasta Palace (Tallinn). The Network tab shows one request: `GET http://localhost:8080/restaurants` returning `200` with a paged JSON body.

screenshots/scene3-step1.png

Figure 7: Frontend at `:8090` after clicking Restaurants: six cards rendered from the live `GET /restaurants` call to the gateway.

DO

In the browser, click on the *Pizza Antonio* card.

SEE

The detail view loads with Pizza Antonio's address (Ruutli 12, Tartu), operating hours (11:00–22:00), and an *Open / Closed* badge. Below it, the menu lists Margherita, Quattro Formaggi, Tiramisu, Garlic Bread, and any items added during Scene 2. The Network tab shows two new calls: `GET /restaurants/d0000001-0000-0000-0000-000000000001` (200) and `GET /restaurants/d0000001-0000-0000-0000-000000000001/menu-items` (200).

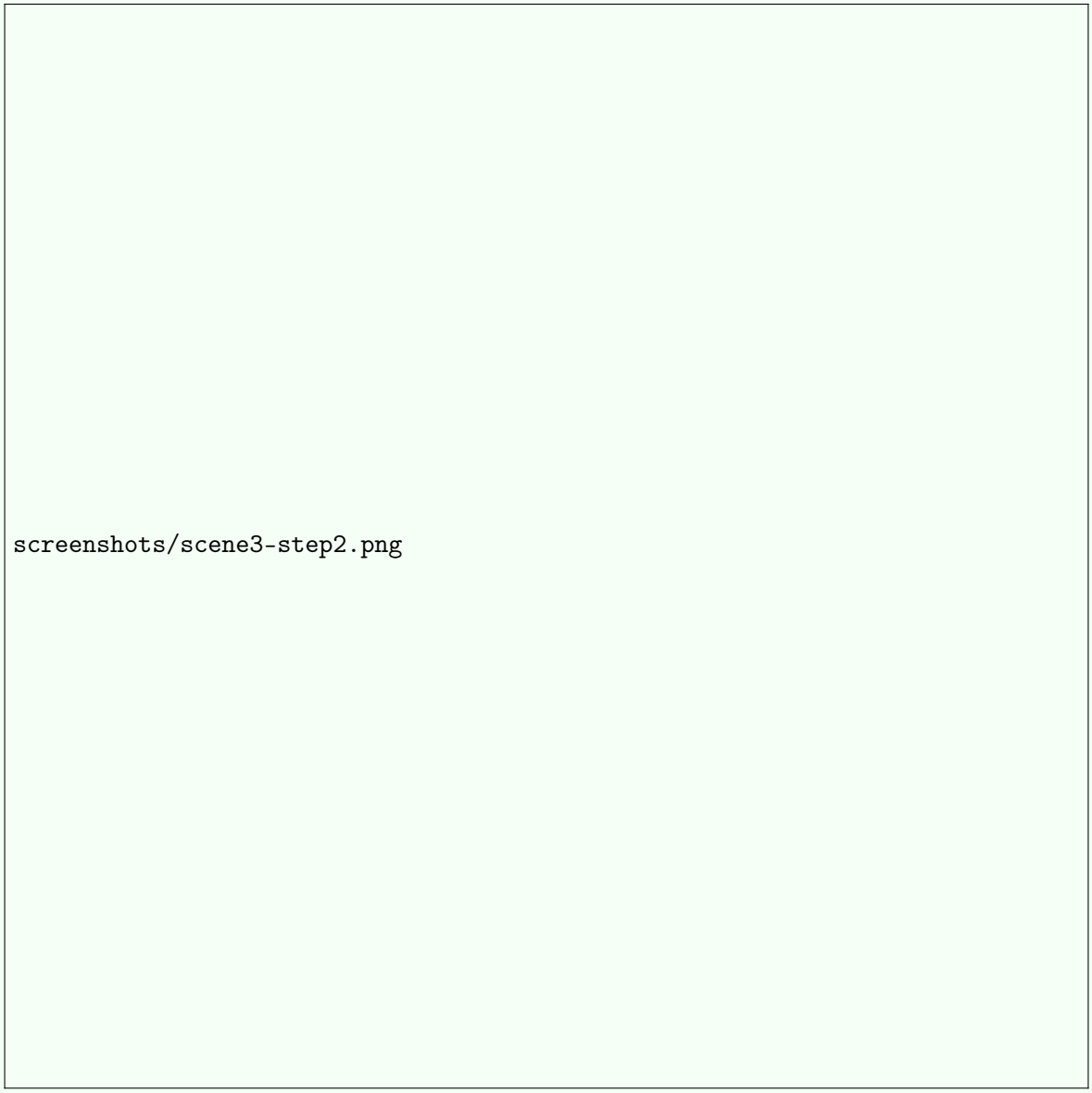

Figure 8: Restaurant detail view: live data from `restaurant-service` and `menu-service`, both fetched through the gateway by the SPA.

PROVES

[BASE] "Frontend: The frontend exists. It calls at least one backend endpoint (per student) and displays real data." *(2 points; Project2026.pdf p. 4.)* The Vue 3 SPA in `frontend/src/views/RestaurantListView.vue` calls `GET /restaurants`; `RestaurantDetailView.vue` additionally calls `GET /restaurants/{id}` and `GET`

`/restaurants/{id}/menu-items`. All three are Sierra-Lima endpoints; the data on screen is fetched live from the running services.

4 Wrap-up

SAY

"That's the Checkpoint 2 deliverable – happy to take questions."

DO

After the instructor confirms they are done, in PowerShell:

```
docker compose down
```

Close the browser tabs and the IntelliJ window.

SEE

`[+] Running 7/7` with each container marked `Removed`. Network `quickbite_quickbite-net` also `Removed`.

5 If something goes wrong

- ◇ **Docker Desktop is unreachable** (error `error during connect: ... open //./pipe/docker_engine`): in a NEW PowerShell window run `wsl --shutdown`, wait five seconds, reopen Docker Desktop, wait until the whale icon stops animating, then re-run the failing command.
- ◇ **Port already allocated** (error `Bind for 0.0.0.0:8080 failed`): in PowerShell run `docker compose down`, then `docker ps` to find any leftover container holding the port, then `docker rm -f <container-id>` on it, then `docker compose up -d`.
- ◇ **A container is unhealthy** after a long wait: in PowerShell run `docker compose logs <service-name> --tail 80` (substitute `restaurant-service`, `menu-service`, `gateway`, or `frontend`), read the last error, then `docker compose restart <service-name>`; re-run `docker compose ps` until it shows `(healthy)`.
- ◇ **The Scene 2 `curl.exe` returns 401 Unauthorized**: the JWT was probably mistyped or split across lines. Re-paste the full `Authorization: Bearer ...` header from this script as a single line (no line breaks inside the token), keeping the backticks at the end of each continuation line.
- ◇ **The Scene 3 frontend page is blank or shows a loading spinner forever**: in DevTools Network tab, look for a failed call to `:8080`. If it shows `(failed) net::ERR_CONNECTION_REFUSED`, the gateway is down – run `docker compose ps gateway` and `docker compose restart gateway`, then refresh the page.
- ◇ **Total wipe and rebuild** (only if nothing above helps): in PowerShell run `docker compose down -v`, then `docker compose up -d --build`, and wait the full 3–5 minutes for it to come back up.